Capstone Report — Automated IT Infrastructure Platform

Oct 20, 2026 · @Landry

This project consolidated nine prior standalone lab projects into a single documented, automated IT infrastructure platform spanning three VMs, and extended it with FreeIPA identity management and three real internal services (Wiki.js, Snipe-IT, Uptime Kuma), with service deployment codified in Ansible and redeployed through a self-hosted GitHub Actions CI/CD pipeline.

Executive Summary

Over a planned 16-day roadmap (actual elapsed time was considerably longer, driven by real infrastructure troubleshooting rather than scope growth), this project took nine previously separate, manually-run lab exercises and consolidated them into one documented, automated platform, then extended it with centralized identity management and three operational internal services.

The three internal services, their Nginx routes, BIND9 records and firewall rule are codified in Ansible and redeployed automatically by a self-hosted CI/CD pipeline on every push to the ansible-lab repository; the VM builds, FreeIPA and the monitoring stack are documented manual installs (see the Installation Guide). The platform's operational readiness was further proven, not just claimed: two real VM-level filesystem corruption incidents (three corruption events across two VMs) were fully diagnosed and recovered, and one deliberate backup/restore drill caught a flaw in the originally planned restore procedure before it could fail during an actual incident.

Problem Statement and Objectives

The IT department of a small organization lacked a centralized, automated, and monitored platform for deploying and managing internal services. Nine prior projects existed only as disconnected lab exercises — no shared documentation, no unified identity layer, and no real production-like services running on top of the infrastructure they built.

Objectives:

- Re-verify and consolidate nine prior infrastructure projects into one coherent, documented platform

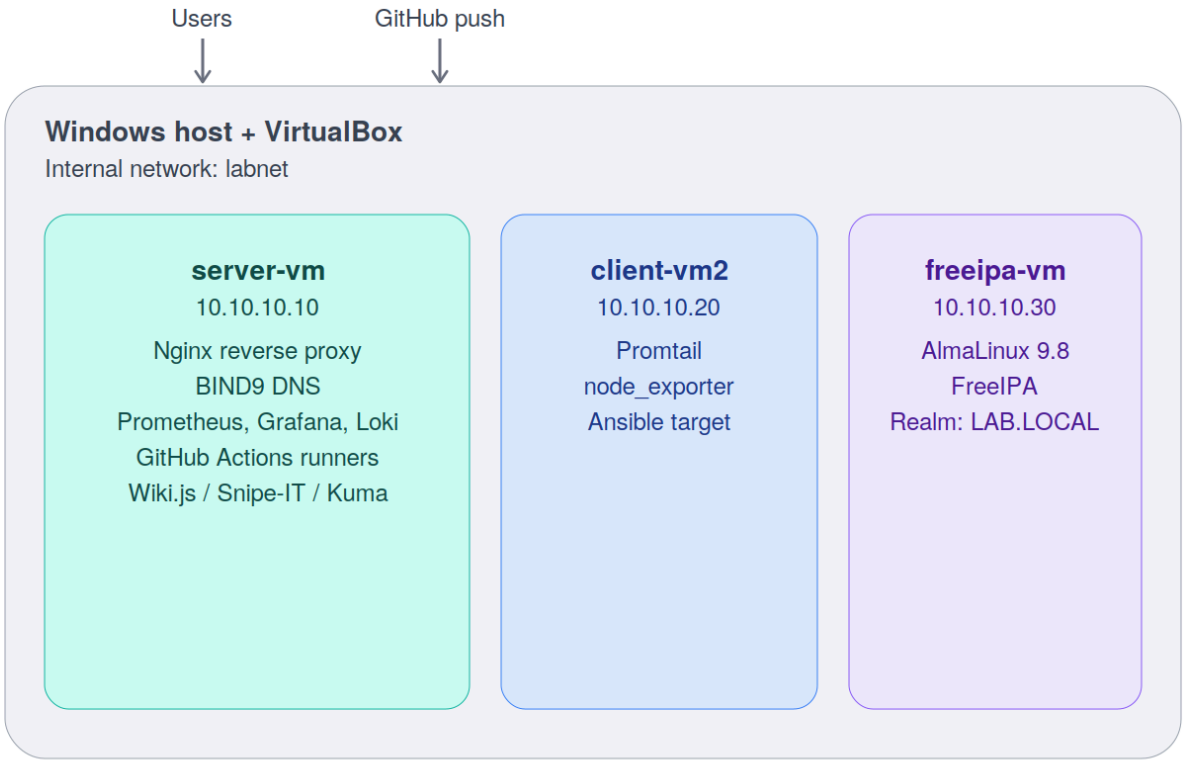
- Add centralized identity management (FreeIPA)
- Deploy three real internal services behind a unified reverse proxy and DNS namespace
- Wire every new service into the existing Prometheus/Grafana/Loki monitoring and logging stack
- Automate the entire deployment with Ansible and a GitHub Actions CI/CD pipeline
- Validate the platform's operational readiness, including disaster recovery, rather than assume it

Scope

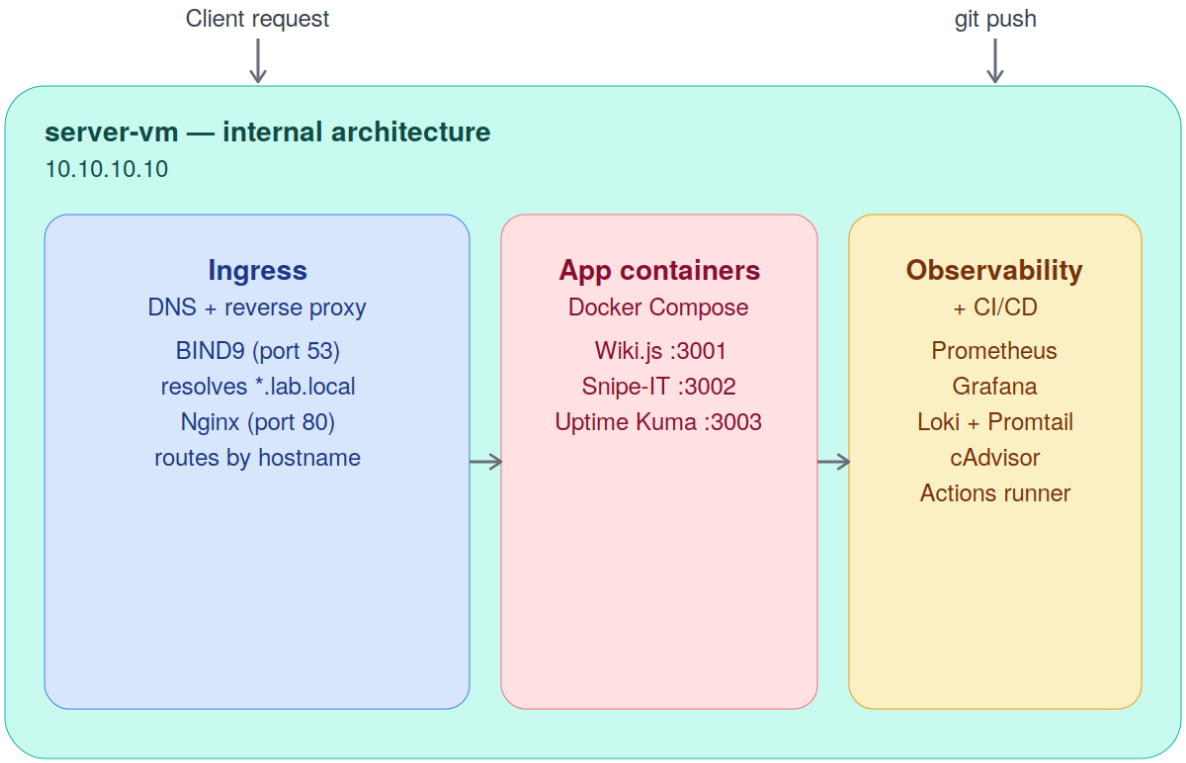
In scope	Out of scope
Ubuntu Server, Docker/Compose	Kubernetes, multi-node clustering
Nginx reverse proxy	Cloud deployment
BIND9 internal DNS	High availability
Prometheus/Grafana/Loki monitoring and logging	Production-grade CA (self-signed/internal CA accepted for this lab)
Ansible automation	SSL/TLS (follow-on project)
GitHub Actions CI/CD	WireGuard VPN (follow-on project)
FreeIPA identity management	Windows Active Directory (follow-on project)
Three internal services: Wiki.js, Snipe-IT, Uptime Kuma	
Full documentation (architecture, administrator, installation, troubleshooting)	

Architecture

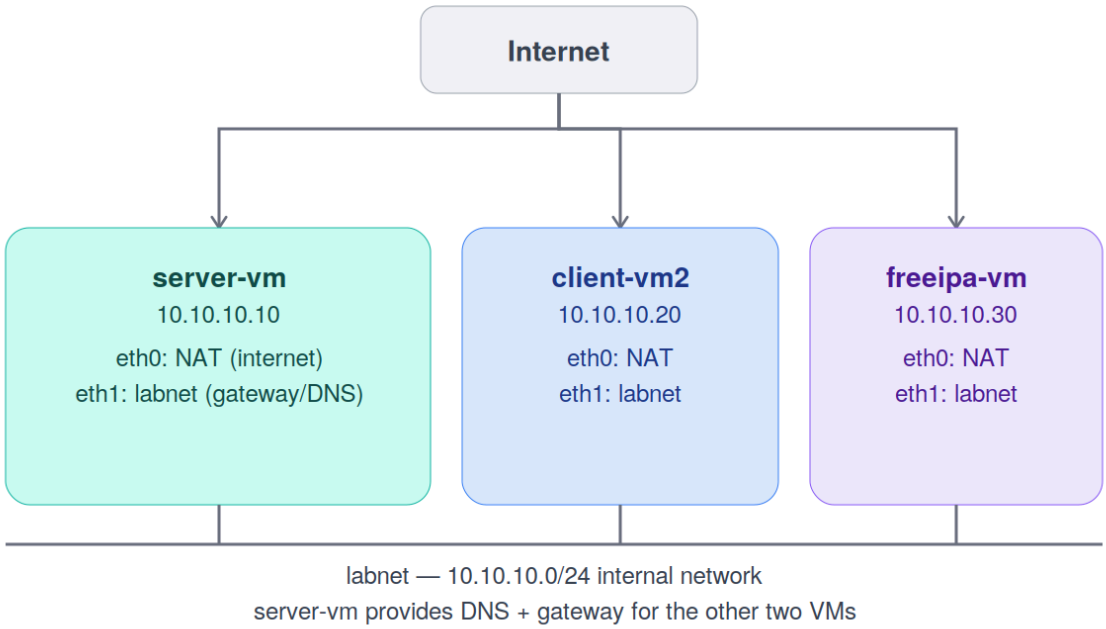
The platform spans three VMs on a single Windows host, connected over a VirtualBox internal network (`labnet`, 10.10.10.0/24), with each VM also carrying its own NAT adapter for independent internet access during package installs and image pulls.



server-vm (10.10.10.10) carries the bulk of the platform: ingress (BIND9 DNS, Nginx reverse proxy), the application containers (Wiki.js, Snipe-IT, Uptime Kuma, each with its own database), and the observability/CI stack (Prometheus, Grafana, Loki/Promtail, cAdvisor, and the GitHub Actions self-hosted runners).



client-vm2 (10.10.10.20) runs the Ansible-managed monitoring agents (node_exporter, Promtail). freeipa-vm (10.10.10.30) runs FreeIPA on AlmaLinux 9.8, standalone (no integrated DNS, to avoid conflicting with BIND9 on server-vm).



Technology Stack

Layer	Technology
OS / networking	Ubuntu Server, static IP, SSH, UFW
DNS	BIND9
Containers	Docker, Docker Compose
Reverse proxy	Nginx
Metrics	Prometheus, Grafana, node_exporter, cAdvisor
Logging	Loki, Promtail
Identity	FreeIPA (AlmaLinux 9.8)
Automation	Ansible
CI/CD	GitHub Actions (self-hosted runners)
Services deployed	Wiki.js (Postgres), Snipe-IT (MariaDB), Uptime Kuma

Implementation Summary

Phase	Deliverable	Status
Day 1	Infrastructure audit of all 9 prior projects (resolved stale container, under-resourced VM, hardcoded Grafana password)	Complete
Day 2-3	Hardening (folded into Day 1: secrets to `.env`, resource allocation)	Complete
Day 4-6	FreeIPA identity management on AlmaLinux 9.8, after failed Ubuntu-native and Docker attempts	Complete
Day 7-9	Wiki.js, Snipe-IT, Uptime Kuma deployed via Docker Compose, Nginx, BIND9	Complete

Phase	Deliverable	Status
Day 10	Monitoring integration: cAdvisor/Loki auto-discovery, 3 Uptime Kuma monitors, DNS and redirect fixes	Complete
Day 11	Ansible consolidation (`capstone-services.yml`), real deploy validated after a disk-corruption recovery	Complete
Day 12	CI/CD extension: second self-hosted runner, `deploy-capstone.yml` workflow, secrets-based `.env` reconstruction	Complete
Backup/restore drill	End-to-end validation of the Wiki.js database restore procedure; corrected a flaw in the originally documented restore command	Complete
Day 13-14	Architecture diagrams (platform, server-vm internals, network topology) and a consolidated troubleshooting guide	Complete
Day 15-16	Report and presentation	This document and the accompanying deck

Key Incidents and Lessons Learned

The single most important pattern across this project: **host resource starvation causes real, block-level filesystem corruption**, not just cosmetic VM pauses. It happened three times across two VMs. When a VirtualBox guest goes without CPU time too long (host out of RAM, out of disk, or paused mid-write), the guest kernel's soft-lockup watchdog fires and any disk write in flight can be corrupted. Repeated `fsck` passes did not reliably converge. What did work: never resize a disk in place while attached (clone-then-resize instead), never force-close a paused VM, and confirm host headroom before any write-heavy operation.

Other recurring lessons, each documented in full in the repository's audit trail:

Issue	Root cause	Fix
Host disk space exhaustion	Four VMs sharing one Windows drive	Freed space before resizing; learned that `docker system prune` often reclaims 0 bytes on a healthy platform
DNS resolution failures on client-vm2	`netplan` cloud-init config silently overrode BIND9 with public DNS	Corrected `/etc/resolv.conf`, flagged netplan as the durable fix
Nginx "missing" despite earlier project	Component never actually persisted on the live VM	Verify with `systemctl status`, never assume prior work survived
FreeIPA install dead ends	No native Ubuntu package, Docker hits systemd/cgroup conflicts	Installed on AlmaLinux 9 (FreeIPA's native platform) instead
Ansible memory pre-flight check failing on correctly-configured VMs	Threshold set against VirtualBox's allocation, not what the guest OS actually reports (~700MB overhead)	Lowered threshold from 4096MB to 3072MB
Playbook referencing uncommitted files	Files created directly on the VM, never `git add` ed	Validate a playbook against a fresh clone, not just the live server
Backup restore silently no-op	`pg_dump` restore into an already-populated database throws "already exists" errors but changes nothing	Corrected procedure: stop app → drop/recreate database → restore → restart

Full command-by-command detail for every incident above is in `audit/` and the consolidated `docs/troubleshooting-guide.md`.

Validation and Testing

The platform was not considered done once services were reachable — three separate validation passes were run to prove the automation and recovery procedures actually work, not just that they run without erroring:

CI/CD pipeline. A test push to the `ansible-lab` repository triggered the self-hosted GitHub Actions runner end to end: secrets reconstructed into `.env`, `ansible-playbook` executed against `client-vm2`, and the run completed green in the Actions tab.

Ansible validation. `ansible-playbook ... --check` passes cleanly against both `server-vm` and `client-vm2`, and a full non-check run completes with `failed=0`.

Backup/restore drill. A real `pg_dump` backup of the Wiki.js database was taken (97KB, confirmed non-empty), a visible test page was created in the live wiki, and a restore was attempted. The first restore attempt completed without throwing a fatal error but silently failed to actually overwrite the live data — `psql` logged "already exists" for every schema statement because the target database was not empty. The corrected procedure (stop the app, drop and recreate the database, then restore into the empty database, then restart) was re-run and verified: the test page was gone and the wiki reverted to its pre-backup state, proving the restore genuinely works. This caught a real gap between "the documented command runs" and "the data is actually recoverable" before it could matter in a real incident.

Results and Outcomes

- Nine prior lab projects consolidated and re-verified from a fresh state, all issues found resolved
- Three real internal services (Wiki.js, Snipe-IT, Uptime Kuma) deployed, reachable by clean internal hostnames, and monitored
- Centralized identity management (FreeIPA) operational on AlmaLinux 9.8, with Kerberos authentication verified (kinit/klist) and the web UI reachable
- Full observability: every new container auto-discovered by Prometheus/cAdvisor and Loki/Promtail with zero extra config, plus active uptime monitoring on all three services
- Service deployment (Compose stack, Nginx routes, BIND9 records, firewall rule) codified in Ansible and redeployed automatically via CI/CD on every push
- A validated, corrected disaster-recovery procedure, proven end to end rather than assumed
- Two real incidents (three filesystem-corruption events across two VMs) fully diagnosed, recovered, and turned into durable documentation rather than silently worked around

- Four durable documentation artifacts produced: Installation Guide, Administrator Guide, Troubleshooting Guide, and architecture diagrams, in addition to this report

Conclusion and Future Work

This project set out to turn nine disconnected lab exercises into one automated, monitored, documented platform — and it did, while also surviving and documenting real infrastructure failures along the way rather than working in an idealized lab environment. The gap between planned and elapsed time was driven almost entirely by genuine troubleshooting (OS platform mismatches, host resource starvation, DNS misconfiguration, a flawed restore procedure), which is itself the project's strongest evidence: every fix was verified against a real failure, not assumed to work.

Explicitly out of scope, treated as follow-on projects:

- SSL/TLS across all internal services (currently HTTP-only with a self-signed/internal CA acceptable for lab use)
- WireGuard VPN for remote access
- Windows Active Directory, as an alternative or complement to FreeIPA
- Kubernetes/multi-node clustering and high availability, if the platform needed to scale beyond a single-host lab deployment

Recommended next steps if this platform continues to evolve:

- Extend the backup/restore drill to Snipe-IT's MariaDB database (only Wiki.js's Postgres database has been drilled so far)
- Drill a full VirtualBox snapshot restore of server-vm (only the Wiki.js database restore has been exercised so far)
- Script the corrected backup/restore procedure as a single command to remove the chance of an operator skipping the drop/recreate step under incident pressure

Repository and References

- Main platform repository: github.com/Landry5545/capstone-infrastructure-platform
- Ansible playbooks: github.com/Landry5545/ansible-lab
- Prior foundation projects: linux-networking-lab, dockerized-web-infrastructure, nginx-reverse-proxy, custom-dns-server, monitoring-stack
- Installation Guide

- Administrator Guide
- Troubleshooting Guide
- License: MIT